```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor
```

```
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9]])
y = np.array([35, 45, 52, 60, 68, 75, 82, 88, 93])
```

```
model = KNeighborsRegressor(n_neighbors=3)
```

```
model.fit(X, y)
```

```
KNeighborsRegressor
KNeighborsRegressor(n_neighbors=3)
```

```
new_student = np.array([[5.5]])
```

```
prediction = model.predict(new_student)

print("Study Hours:", new_student[0][0])
print("Predicted Mark:", round(prediction[0], 2))
```

```
Study Hours: 5.5
Predicted Mark: 67.67
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([35, 45, 52, 60, 68, 75, 82, 88, 93])
```

```
query = 8.5
```

```
tau = 1.0
```

```
weights = np.exp(-((X - query) ** 2) / (2 * tau ** 2))
```

```
print("Study Hour   Mark   Weight")
print("---------------------------")
```

```
Study Hour   Mark   Weight
---------------------------
```

```
for hour, mark, weight in zip(X, y, weights):
   print(f"{hour:^10} {mark:^6} {weight:.3f}")
```

```
    1        35   0.000
    2        45   0.000
    3        52   0.000
    4        60   0.000
    5        68   0.002
    6        75   0.044
    7        82   0.325
    8        88   0.882
    9        93   0.882
```

```
X_design = np.column_stack((np.ones(len(X)), X))
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([35, 45, 52, 60, 68, 75, 82, 88, 93])
```

```
centers = np.array([2, 5, 8])
```

```
sigma = 1.5
```

```
def rbf(x, center):
   return np.exp(-((x - center) ** 2) / (2 * sigma ** 2))
```

```
features = []

for x in X:
   features.append([rbf(x, c) for c in centers])

features = np.array(features)
```

```
model = LinearRegression()
model.fit(features, y)
```

```
▾ LinearRegression
LinearRegression()
```

```
query = 5.5
```

```
query_features = np.array(
   [[rbf(query, c) for c in centers]]
)
```

```
prediction = model.predict(query_features)

print("RBF Centers:", centers)

print("\nActivation of each center:")
```

```
RBF Centers: [2 5 8]

Activation of each center:
```

```
for center, activation in zip(centers, query_features[0]):
 print(
"Center:",
center,
"Activation:",
 round(activation, 3)
)
```

```
Center: 2 Activation: 0.066
Center: 5 Activation: 0.946
Center: 8 Activation: 0.249
```

```
print("\nStudent Study Hours:", query)
print("Predicted Mark:", round(prediction[0], 2))
```

```

Student Study Hours: 5.5
Predicted Mark: 72.28
```

```
cases = [
 {"hours": 2, "mark": 45},
 {"hours": 4, "mark": 60},
 {"hours": 6, "mark": 75},
{"hours": 8, "mark": 88}
]
```

```
new_hours = 5.5
```

```
best_case = None
smallest_distance = float("inf")
```

```
 print(
"Previous Case:",
 case["hours"],
 "hours | Distance:",
 distance
)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_1593/2121528049.py in <cell line: 0>()
      1 print(
      2 "Previous Case:",
----> 3 case["hours"],
      4 "hours | Distance:",
      5 distance

NameError: name 'case' is not defined
```